

bwtrack: Per-Process Bandwidth Tracking with eBPF

Project T001

Kshitij Kasodkar (23110175) | Hanamanthagouda Policepatil (23110126) | Manav Mangal Jain (22110140)
Thipparapu Rushitha (23110338) | Ralebhat Priyanka Shriram (24110278)

Guide: Prof. Sameer Kulkarni | TA: Jainish, Shubham
IIT Gandhinagar

1 · PROBLEM & OBJECTIVES Who is using the network, and what is lost on the way?

- Linux counts traffic **per interface** (/proc/net/dev). It does not say **which program** sent it.
- Capture tools (tcpdump, nethogs) see packets after they left the program and **guess** the owner from /proc: short connections are missed and every packet costs work.
- For TCP, “bytes the app sent” and “bytes on the wire” are **different numbers**: lost segments are resent and data waits in buffers. A real-time number cannot simply be “100 % accurate”; we must say which number and how it differs.

Our approach: count inside the kernel with eBPF at the moment the process hands data to TCP/UDP (the kernel still knows the PID there), and compare every number with independent ground truth.

Objective	How we verified it (Raspberry Pi 5)
O1 Attribute every byte to its process and remote endpoint: TCP, UDP, IPv4, IPv6	9 correctness tests vs. bytes printed by the workload itself: **9/9**
O2 Run live on real hardware, no mock data	Pi 5, kernel 6.18, 13 kernel probes attached
O3 Show what the network does: retransmissions, RTT, UDP loss	Pi ↔ laptop over Wi-Fi, tc netem 0–10 % loss, 3 repeats, +20 ms delay run
O4 Be honest about real-time accuracy	3 layers (app, TCP wire, NIC) vs. truth from both ends
O5 Low overhead, scalable, stable	OFF-ON-OFF throughput, 1-200 connections, 30-min soak

Evidence: Code/experiments/pi_results/2026-10-30_final/ · proof suite proof_*/summary.md · loss_run1..3, delay20

2 · ARCHITECTURE Count where the process still owns the data

PROCESS CONTEXT: KNOWS THE PID

`tcp_sendmsg`
kprobe + kretprobe: return value = bytes really accepted

`tcp_cleanup_rbuf`
kprobe: advance of copied_seq = bytes read (FIN excluded)

`udp(v6)_sendmsg`
kprobe + kretprobe: destination from msg_name

`skb_consume_udp`
kprobe: UDP bytes read

Runs inside the app's own send()/recv() system call, so bpf_get_current_pid_tgid() is the true owner.

KERNEL: eBPF PROGRAMS WRITE INTO MAPS

`flows (LRU hash)`
socket, PID, protocol, IP:port → app bytes sent/read + final wire bytes of closed sockets

`tcp_snaps`
copy of the kernel's own tcp_sock counters: wire bytes, retransmitted, acked, RTT, cwnd

`sk_owner · udp_drops · perf buffer`

▲ network side (softirq: look up the socket's owner)

`tcp_rcv_established` refresh ≤ 100 ms ·
`tcp_retransmit_skb` every retransmission ·
`__udp_enqueue_schedule_skb` UDP drops

`inet_sock_set_state`: counters start at 0 at SYN; at close the final counters are added to the flow **NEW**

USER SPACE (PYTHON)

Reads all maps every 0.5–1 s (bpf() syscall): no per-packet copying

Δ counter ÷ Δ time = rate per process / remote / protocol

3 layers side by side: app (socket) · TCP wire · NIC (/proc/net/dev)

Terminal UI · web dashboard · SQLite history · Prometheus

BCC compiles the C program against the running kernel; compat.py patches BCC's old headers on 6.18

Evidence: Code/bwtracker/bpf/tracker.bpf.c · doctor.txt (13 probes attached on kernel 6.18.50) · screenshots P4

3a · EXTENSION & ISSUES FIXED Rebuilt from scratch; every fix has a test that proves it

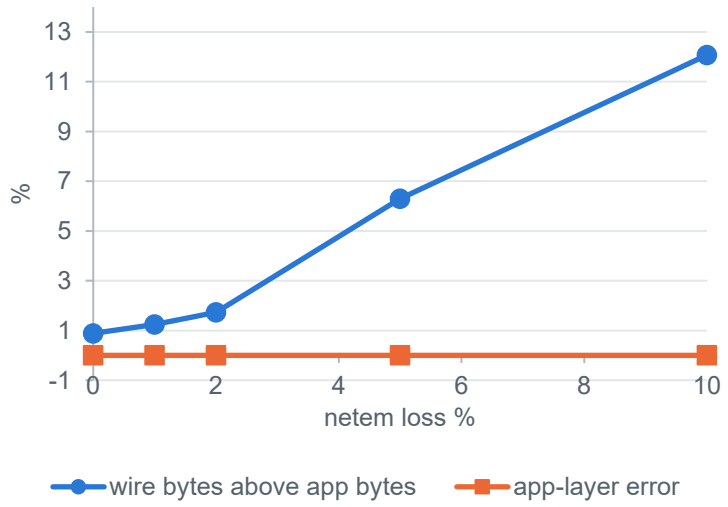
Built on top: real kernel data only (mock removed) · app vs. TCP-wire vs. NIC accounting · retransmissions, RTT, UDP drops per process · Raspberry Pi 5 support with automatic header fix · live terminal + web UI, SQLite history, Prometheus · one-command experiment and proof scripts

Issue	Fix	Proof on the Pi (truth = bytes printed by the program itself)
Non-blocking send() accepts fewer bytes than asked	count the kretprobe return value, not the requested size	15,535,328 B = accepted **exact**; counting the request would say +108 %
Short connections showed 0 wire bytes **[found by our tests]**	at close, the kernel adds the socket's final counters to its flow	500 connections: wire 10,000,000 / 10,000,000 B **exact**
End-of-stream FIN counted as 1 byte of data **[found by our tests]**	don't count the FIN's sequence number	MSG_WAITALL receive 20,971,520 B **exact**
First chunk of each connection missing from the wire count	start TCP counters from 0 at SYN	wire +0.000 % at 0 % loss (before: 19 B short)
Unconnected UDP sendto() to many hosts	read the destination from msg_name	3 IPs, 300 / 600 / 900 kB each **exact**, nothing to 0.0.0.0
IPv6 and IPv4-mapped UDP counted twice (nested calls)	count only the outer call	TCP ::1 10,485,760 B; UDP v6 and v4-mapped 500,000 B **exact**
Threads, fork(), the host's own IP	group by process (tgid); detect own IPs	8 threads → 1 PID; 4 children each own PID; own LAN IP marked local: **all pass**
Pi kernel 6.18 newer than BCC's headers: compile error	compat.py copies the missing definitions from the kernel	3 definitions added, compiles in 5–9 s, 13 probes

Evidence: pi_results/2026-10-30_final/proof_20261030-231542/: partial_sends, short_connections_reuse, recv_waitall, udp_unconnected_multi_dest, ipv6 (.json) · screenshot P1

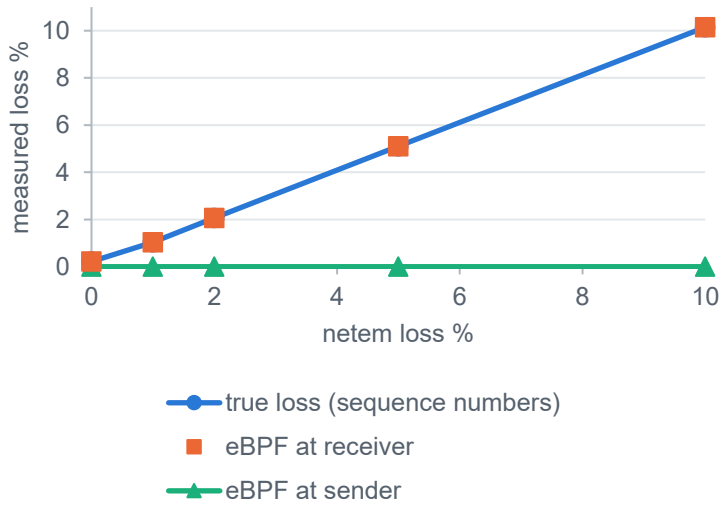
3b · EVALUATION App bytes stay exact under loss; wire and instant rates do not

TCP upload (mean of 3 runs)



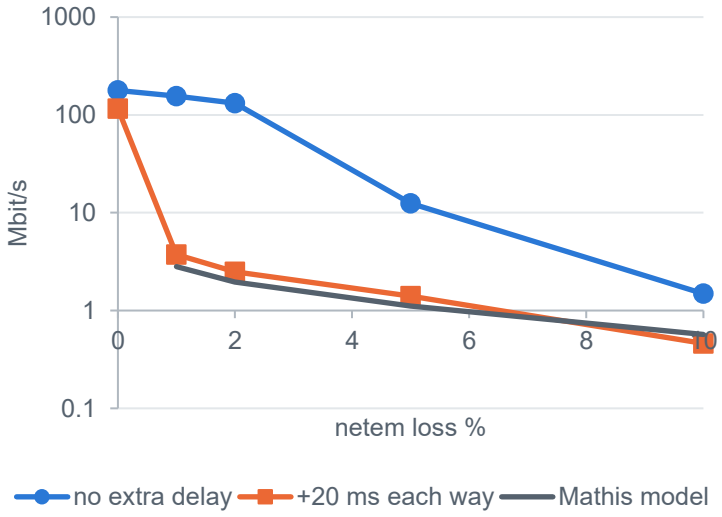
App-layer bytes = truth in every trial. Wire bytes grow with loss (retransmissions): +12.1 % at 10 %.

UDP loss (mean of 3 runs)



The sender's eBPF can't see UDP loss. At the receiver, eBPF loss = true loss in all 15 trials.

TCP upload goodput, Mbit/s (log)



+20 ms delay: 1 % loss cuts upload from 156 to 3.7 Mbit/s, following TCP theory (Mathis) with the RTT our tool measured.

59 / 59

completed loss trials with app-layer bytes exactly equal to truth (one 10 % upload timed out in our generator)

29-90 %

mean gap between app rate and wire rate per 0.5 s sample. Totals are exact, instants are not: at 10 % loss the wire trails the app by 9–27 s

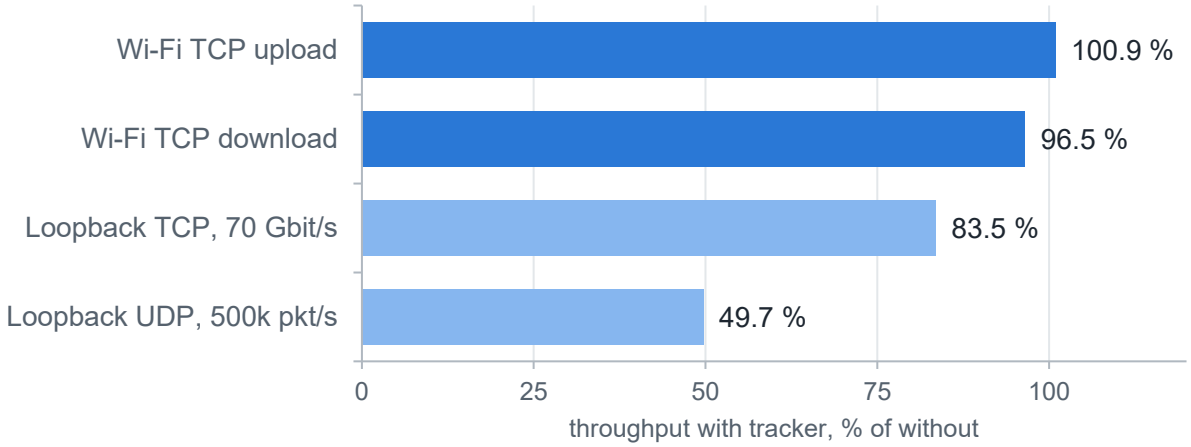
2985 = 2985

UDP receive-buffer drops: kernel counter vs. our eBPF counter

Evidence: loss_combined/combined.md, delay20/*/report.md · setup: Pi 5 ↔ laptop over hotspot Wi-Fi, netem both directions (IFB), mean of 3 runs · screenshots P2, P2b

4 · NON-FUNCTIONAL TESTING Measured, not assumed: overhead, scale, stability

Performance: OFF → ON → OFF, CPU cores pinned, 4 runs each



- **Real Wi-Fi: no measurable change** (upload 174.6 → 176.2 Mbit/s; download within Wi-Fi noise).
- Cost shows only at extreme rates: 70 Gbit/s loopback −16.5 %, 500k tiny UDP packets/s −50 % (≈ 0.3–0.6 µs per send).
- eBPF **294 ns** average per run over 7.3 M runs · tracker process **0.29 % CPU** · maps 11.4 MB · RSS 220 MB (BCC/LLVM compiler stays loaded).

Scalability

- 1, 10, 50, 100, 200 simultaneous connections: **every one exact**; collector CPU 4.0 → 12.4 %
- 170,388 tiny send() calls/s: 87,238,848 B **exact**
- 50 processes at once: 49/50 in the suite, then **250/250** in 5 repeats

Reliability

- 30-min soak of the real recorder: **2511 / 2511** short-lived processes exact in SQLite; RSS 220.3 → 223.3 MB; CPU 1.9 %
- 0 kretprobe misses, 0 lost events in all tests
- Map overfilled on purpose: LRU evicts, no crash · started mid-transfer: no false spike (2.03 vs 2.00 MB/s)

Portability

Kernels 5.15 (WSL2), 6.8 (Ubuntu), 6.18 (Pi) · x86-64 and arm64 · BCC 0.18-0.31

Evidence: overhead_20261031-005043.json · proof_*/: ebpf_cost, concurrent_*, event_rate, soak, map_overflow_lru, start_mid_transfer · diag_processes_*.json · P1, P3

5 · CHALLENGES FACED What was hard, and what we did about it

Challenge	What we did	Result
Kernel versions. Pi kernel 6.18 is newer than BCC's bundled headers; no BTF on the Pi, so no CO-RE	Compile at start-up with BCC; compat.py reads the compile errors and copies the missing definitions from the running kernel	Same code on 5.15, 6.8 and 6.18; 5 s start-up, but 220 MB memory
Getting the PID right. Network-side hooks run in softirq, where the “current process” is random	Take the PID only in system-call context; network hooks look up the socket's owner in a map	Threads, forks and 200 connections all attributed exactly
Bugs that only real tests reveal	Built a proof suite where each workload prints its own byte count, then compared	Found and fixed: 0 wire bytes for short connections, FIN counted as data
A tracepoint can be skipped if it fires while another eBPF program runs on that CPU	Fallback: when a socket address is reused, retire the old socket's last snapshot	500 / 500 short connections exact
Measuring overhead honestly. Without the tracker, loopback speed jumped between 26 and 70 Gbit/s	Pin iperf3 to fixed cores; OFF → ON → OFF; also measure the real Wi-Fi path	Spread ±1 %; real network shows no change
Creating loss in both directions. netem only shapes outgoing traffic; at 10 % loss TCP backs off for minutes	IFB device for incoming traffic; 3 repeats; longer generator timeout	One 10 % point has 2 runs instead of 3 (we report n)

Known limits we report: a flood of tiny packets costs throughput (-50 % at 500k packets/s); a 49/50 miss seen once was not reproduced in 250 further tries; per-sample rates lag the wire under heavy loss.

Evidence: Code/bwtracker/compat.py · run_final.log · overhead_*.json · Code/docs/ARCHITECTURE.md

PROOF P1 Proof suite on the Raspberry Pi (30 Oct, 23:15)

```
 load), 0.20 % idle; RSS 220.5 MB; accuracy at full speed 100.000 %, 100.000 %, 100.000 %

loading tracker ...
   Compiling eBPF program against the running kernel's headers (BCC/clang)...
   Compiled in 5.0s. Attaching probes...
   Attached 13 probes, skipped 2 optional

== correctness (issues fixed) ==
  PASS partial_sends: tracker 15535328 B = accepted 15535328 B; counting the requested size (old method) would report 32374784 B (+
108 %)
  PASS recv_waitall: receiver with MSG_WAITALL: tracker 20971520 B, app 20971520 B
  PASS sendfile: sendfile(): tracker 20971520 B, app 20971520 B
  PASS udp_unconnected_multi_dest: one sendto() socket -> 3 IPs: 127.0.0.2 300000/300000, 127.0.0.3 600000/600000, 127.0.0.4 900000
/900000
  PASS ipv6: TCP ::1 10485760/10485760, UDP ::1 500000/500000, UDP v4-mapped 500000/500000 (not double counted)
  PASS threads_one_pid: 8 threads -> one PID: 41943040/41943040 B, 8 sockets
  PASS fork_separate_pids: parent + 4 forked children: 4/4 children exact, parent 1048576/1048576
  PASS short_connections_reuse: 500 short connections: app 10000000/10000000 B, wire 10000000 B (100.00 %)
  PASS loopback_detection: 127.x flagged: True; own IP 10.58.105.107 flagged: True

== scalability ==
       1 connections: 1/1 exact, collector CPU 4.0 %
      10 connections: 10/10 exact, collector CPU 6.81 %
      50 connections: 50/50 exact, collector CPU 5.96 %
     100 connections: 100/100 exact, collector CPU 7.22 %
     200 connections: 200/200 exact, collector CPU 12.43 %
  PASS concurrent_connections: 1 conns: 1/1 exact; 10 conns: 10/10 exact; 50 conns: 50/50 exact; 100 conns: 100/100 exact; 200 conn
s: 200/200 exact
  FAIL concurrent_processes: 50 processes at once: 49/50 attributed exactly
  PASS event_rate: 170,388 send() calls/s of 64 B: tracker 87238848/87238848 B; send probes on_tcp_send_enter 143.1 ns, on_tcp_send
_return 603.6 ns
  ebpf_cost_all_tests: 7,338,783 eBPF runs, avg 294 ns; maps 11.4 MB; kretprobe misses 0, perf lost 0

== reliability ==
  PASS map_overflow_lru: 600 UDP flows: default map 100.0 % captured; forced 256-entry map: tool alive=True, 0.6 % captured (LRU ev
icts, memory stays bounded, no crash)
  PASS start_mid_transfer: tracker started 5 s into a 2 MB/s transfer: max reported 2.03 MB/s (no spike from earlier bytes)
```

Evidence: run_final.log, proof_20261030-231542/summary.md (30-min soak: 2511/2511 exact, RSS 220.3 → 223.3 MB)

PROOF P2 Loss experiment, run 2 of 3: every trial compared with ground truth

```
--- UDP receive-buffer overflow test ---
  rcvbuf   kernel drops 2985  eBPF drops 2985  rx accuracy 100.0%

Wrote experiments/results/final_20261030-2315/loss_run1/20261030-235212/report.md, report.html, results.csv, series.csv

== loss experiment run 2/3 ==
Loading eBPF tracker... (iface wlan0, peer 10.58.105.150)
   Compiling eBPF program against the running kernel's headers (BCC/clang)...
   Compiled in 5.0s. Attaching probes...
   Attached 13 probes, skipped 2 optional

--- netem loss 0.0% delay 0.0 ms on wlan0 (both) ---
  tcp-up   app accuracy 100.0%  wire +0.000%  retx 0.0% (0 segs)  goodput 185.127 Mbit/s  rtt 31.3 ms  wire lag 0.0 s
  tcp-down app accuracy 100.0%  goodput 119.653 Mbit/s
  udp-up   sender accuracy 100.0%  true loss 0.0%  (sender-side eBPF cannot see it)
  udp-down rx accuracy 100.0%  true loss 0.0%  eBPF-measured loss 0.0%

--- netem loss 1.0% delay 0.0 ms on wlan0 (both) ---
  tcp-up   app accuracy 100.0%  wire +0.621%  retx 0.618% (90 segs)  goodput 178.824 Mbit/s  rtt 27.96 ms  wire lag 0.0 s
  tcp-down app accuracy 100.0%  goodput 16.146 Mbit/s
  udp-up   sender accuracy 100.0%  true loss 1.123%  (sender-side eBPF cannot see it)
  udp-down rx accuracy 100.0%  true loss 1.018%  eBPF-measured loss 1.018%

--- netem loss 2.0% delay 0.0 ms on wlan0 (both) ---
  tcp-up   app accuracy 100.0%  wire +1.705%  retx 1.677% (247 segs)  goodput 142.216 Mbit/s  rtt 11.91 ms  wire lag 0.5 s
  tcp-down app accuracy 100.0%  goodput 13.016 Mbit/s
  udp-up   sender accuracy 100.0%  true loss 1.843%  (sender-side eBPF cannot see it)
  udp-down rx accuracy 100.0%  true loss 2.112%  eBPF-measured loss 2.112%

--- netem loss 5.0% delay 0.0 ms on wlan0 (both) ---
  tcp-up   app accuracy 100.0%  wire +6.090%  retx 5.74% (882 segs)  goodput 16.101 Mbit/s  rtt 7.26 ms  wire lag 1.5 s
  tcp-down app accuracy 100.0%  goodput 8.708 Mbit/s
  udp-up   sender accuracy 100.0%  true loss 5.117%  (sender-side eBPF cannot see it)
  udp-down rx accuracy 100.0%  true loss 4.935%  eBPF-measured loss 4.935%

--- netem loss 10.0% delay 0.0 ms on wlan0 (both) ---
```

Evidence: runs 1–3: loss_run1..3/*/report.html

PROOF P2b Packet loss at its worst (10 %), and with 20 ms extra delay

```
--- netem loss 10.0% delay 0.0 ms on wlan0 (both) ---
  tcp-up   app accuracy 100.0%  wire +12.111%  retx 10.802% (1755 segs)  goodput 1.812 Mbit/s  rtt 23.13 ms  wire lag 9.01 s
  tcp-down app accuracy 100.0%  goodput 6.439 Mbit/s
  udp-up   sender accuracy 100.0%  true loss 9.254%  (sender-side eBPF cannot see it)
  udp-down rx accuracy 100.0%  true loss 10.022%  eBPF-measured loss 10.022%

Wrote experiments/results/final_20261030-2315/loss_run2/20261030-235752/report.md, report.html, results.csv, series.csv
```

Run 2 at 10 % loss: app bytes still exact; wire +12.1 % (1755 retransmitted segments seen live by eBPF); UDP loss at the receiver = true loss (10.022 %); the sender's eBPF cannot see UDP loss.

```
== loss experiment with 20 ms extra delay ==
Loading eBPF tracker... (iface wlan0, peer 10.58.105.150)
   Compiling eBPF program against the running kernel's headers (BCC/clang)...
   Compiled in 5.1s. Attaching probes...
   Attached 13 probes, skipped 2 optional

--- netem loss 0.0% delay 20.0 ms on wlan0 (both) ---
  tcp-up   app accuracy 100.0%  wire +0.000%  retx 0.0% (0 segs)  goodput 115.704 Mbit/s  rtt 70.57 ms  wire lag 0.5 s
  tcp-down app accuracy 100.0%  goodput 130.939 Mbit/s

--- netem loss 1.0% delay 20.0 ms on wlan0 (both) ---
  tcp-up   app accuracy 100.0%  wire +1.049%  retx 1.039% (152 segs)  goodput 3.741 Mbit/s  rtt 50.26 ms  wire lag 0.5 s
  tcp-down app accuracy 100.0%  goodput 3.077 Mbit/s

--- netem loss 2.0% delay 20.0 ms on wlan0 (both) ---
  tcp-up   app accuracy 100.0%  wire +2.196%  retx 2.148% (318 segs)  goodput 2.498 Mbit/s  rtt 51.19 ms  wire lag 1.0 s
  tcp-down app accuracy 100.0%  goodput 2.45 Mbit/s
```

+20 ms delay each way: 1 % loss drops upload from 115.7 to 3.7 Mbit/s; app accuracy stays 100 %; retransmissions grow with loss (152 → 318 segments).

Evidence: loss_run2/*/report.html, delay20/*/report.html, run_final.log

PROOF P3 Overhead check (OFF → ON → OFF) and the 50-process follow-up

```
# Throughput with vs without the tracker (pinned cores, OFF-ON-OFF)

Raspberry Pi 5 Model B Rev 1.1, kernel 6.18.50+rpt-rpi-2712. 4 runs per block. Tracker process CPU while ON: 0.29 %, RSS 220.5 MB.

| workload | without (mean ± sd, n) | with (mean ± sd, n) | change of mean | change of median |
|---|---|---|---|---|
| tcp_loopback_gbps | 69.468 ± 0.998 (8) | 58.008 ± 0.939 (4) | -16.5 % | -16.4 % |
| udp64_loopback_kpps | 500.134 ± 2.706 (8) | 248.356 ± 0.648 (4) | -50.3 % | -50.4 % |
| wifi_tcp_up_mbps | 174.599 ± 8.499 (8) | 176.205 ± 9.077 (4) | +0.9 % | +0.5 % |
| wifi_tcp_down_mbps | 125.083 ± 13.26 (8) | 120.741 ± 17.762 (4) | -3.5 % | +2.8 % |

saved /home/kshitij/bwtracker/experiments/results/overhead_20261031-005043.json
```

Real Wi-Fi +0.9 % / −3.5 % (within noise); loopback stress −16.5 % / −50 %. Tracker process 0.29 % CPU.

```
kshitij@kasodkar:~/bwtracker $ cd ~/bwtracker
kshitij@kasodkar:~/bwtracker $ sudo python3 experiments/proof/diag_processes.py
[sudo] password for kshitij:
round 1: 50 procs ran, exact early 50, late 50, cumulative 50, worker errors 0
round 2: 50 procs ran, exact early 50, late 50, cumulative 50, worker errors 0
round 3: 50 procs ran, exact early 50, late 50, cumulative 50, worker errors 0
round 4: 50 procs ran, exact early 50, late 50, cumulative 50, worker errors 0
round 5: 50 procs ran, exact early 50, late 50, cumulative 50, worker errors 0
kprobe misses 0 perf lost 0
saved /home/kshitij/bwtracker/experiments/results/diag_processes_20261031-003010.json
```

The one FAIL (49/50) re-tested: 5 rounds × 50 processes, all exact; 0 kprobe misses.

```
kshitij@kasodkar:~/bwtracker $ sudo ./bwtrack report --since 2h
Top programs (2026-10-30 22:41 -> now), loopback excluded
PROGRAM          PIDS        SENT   RECEIVED      TOTAL  TCP RETX UDP DROP  REMOTE IPS
curl             1         1.8 KB    47.8 MB    47.8 MB     0.00%        -  2606:4700:7::da, 10.58.105.124
sshd-session     3        29.8 KB      643 B    30.5 KB     0.74%        -  10.58.105.150
sshd-auth        1         2.9 KB     3.6 KB     6.5 KB     0.00%        -  10.58.105.150
(shown rows total)        34.6 KB    47.8 MB    47.8 MB
```

SQLite history after a real download: curl 47.8 MB, IPv6 to Cloudflare.

Evidence: overhead_20261031-005043.json, diag_processes_20261031-003010.json

PROOF P4 Live tracking with real traffic, compile on kernel 6.18, laptop peer

```
bwtrack - eBPF per-process bandwidth   00:39:47   interval 1.00s
Apps (socket layer)  up     1.8 kbit/s  down     7.6 Mbit/s   | TCP wire up    1.8 kbit/s  retrans 0.00%  UDP drops 0
NIC (ground truth)   wlan0: up 169.3 kbit/s down 8.0 Mbit/s
tracker cpu 0.0%  sockets 6  kprobe misses 0  perf lost 0
view: PROCESSES   proto: ALL   sort: total rate   loopback: hidden (0)   idle: hidden
[p]rocesses [f]lows [t]proto [s]ort [l]oopback [i]dle [e]vents [q]uit

    PID PROCESS         PROTO   REMOTES   UP app/s  DOWN app/s  UP wire/s  RETX%     RTT RETX segs UDP drop  TOTAL UP
  24307 curl            TCP/UDP       2        0 B/s   926.4 KB/s        0 B/s      -   63.4ms         -        -    1.8 KB
   1277 sshd-session    TCP           1      220 B/s       0 B/s      220 B/s   0.0%    8.5ms         2        -   15.1 KB
recent events (retransmissions = packet loss on the path) ------------------------------------------------------------
00:39:30 RETRANS  sshd-session(1277)     TCP 10.58.105.107:22 -> 10.58.105.150:4700  retransmitted 84 B
00:39:30 RETRANS  sshd-session(1277)     TCP 10.58.105.107:22 -> 10.58.105.150:4700  retransmitted 132 B
```

bwtrack live during a 50 MB download: curl 926 KB/s vs NIC 8.0 Mbit/s; real Wi-Fi retransmissions on the SSH session appear as events.

```
Trial compile + attach of the real eBPF program...
    Compiling eBPF program against the running kernel's headers (BCC/clang)...
      BCC's bpf.h is older than this kernel's headers; adding 3 definition(s) from the kernel
    Compiled in 9.3s. Attaching probes...
    Attached 13 probes, skipped 2 optional
    attached kprobe     tcp_sendmsg
    attached kretprobe  tcp_sendmsg
    attached kprobe     tcp_cleanup_rbuf
    attached kprobe     tcp_rcv_established
    attached kprobe     udp_sendmsg
    attached kretprobe  udp_sendmsg
    attached kprobe     udpv6_sendmsg
    attached kretprobe  udpv6_sendmsg
    attached kprobe     skb_consume_udp
    attached kprobe     __udp_enqueue_schedule_skb
    attached kretprobe  __udp_enqueue_schedule_skb
    attached tracepoint tcp:tcp_retransmit_skb
    attached tracepoint sock:inet_sock_set_state
    skipped  kprobe     tcp_sendpage: not in this kernel
    skipped  kretprobe  tcp_sendpage: not in this kernel
[ OK ] eBPF program loads and attaches

All checks passed. Start with:  sudo ./bwtrack live
```

doctor: 3 missing definitions added, 13 probes attached.

```
00:02:58 10.58.105.107 udp-down {'mode': 'udp-down', 'session': 9000312984192246953, 'rate': 20000000.0, 'seconds': 5.0, 'size': 12
00}
session error: peer closed
00:03:13 10.58.105.107 tcp-up {'mode': 'tcp-up'}
00:03:16 10.58.105.107 tcp-down {'mode': 'tcp-down', 'bytes': 20971520}
00:03:18 10.58.105.107 udp-up {'mode': 'udp-up', 'session': 7857147549393600850, 'rate': 20000000.0, 'seconds': 5.0, 'size': 1200}
00:03:26 10.58.105.107 udp-down {'mode': 'udp-down', 'session': 7524465159335834034, 'rate': 20000000.0, 'seconds': 5.0, 'size': 12
00}
00:03:34 10.58.105.107 tcp-up {'mode': 'tcp-up'}
00:03:36 10.58.105.107 tcp-down {'mode': 'tcp-down', 'bytes': 20971520}
00:03:49 10.58.105.107 udp-up {'mode': 'udp-up', 'session': 7676203071384277311, 'rate': 20000000.0, 'seconds': 5.0, 'size': 1200}
00:03:56 10.58.105.107 udp-down {'mode': 'udp-down', 'session': 13950300188404019233, 'rate': 20000000.0, 'seconds': 5.0, 'size': 1
200}
00:04:04 10.58.105.107 tcp-up {'mode': 'tcp-up'}
00:04:07 10.58.105.107 tcp-down {'mode': 'tcp-down', 'bytes': 20971520}
00:04:20 10.58.105.107 udp-up {'mode': 'udp-up', 'session': 14183108569582350877, 'rate': 20000000.0, 'seconds': 5.0, 'size': 1200}
00:04:27 10.58.105.107 udp-down {'mode': 'udp-down', 'session': 15207383299027626088, 'rate': 20000000.0, 'seconds': 5.0, 'size': 1
200}
00:04:35 10.58.105.107 tcp-up {'mode': 'tcp-up'}
00:04:53 10.58.105.107 tcp-down {'mode': 'tcp-down', 'bytes': 20971520}
00:05:10 10.58.105.107 udp-up {'mode': 'udp-up', 'session': 16048620837773356730, 'rate': 20000000.0, 'seconds': 5.0, 'size': 1200}
00:05:17 10.58.105.107 udp-down {'mode': 'udp-down', 'session': 12045118306461216257, 'rate': 20000000.0, 'seconds': 5.0, 'size': 1
200}
00:05:25 10.58.105.107 tcp-up {'mode': 'tcp-up'}
00:07:42 10.58.105.107 tcp-down {'mode': 'tcp-down', 'bytes': 20971520}
00:08:10 10.58.105.107 udp-up {'mode': 'udp-up', 'session': 12530196772332802896, 'rate': 20000000.0, 'seconds': 5.0, 'size': 1200}
```

Laptop running trafficgen.py server: one line per trial requested by the Pi.